

# Архитектура софтверског система

Тим: Стручњаци

Чланови тима:

- Вељко Цоцојевић [veljko.cocojevic@elfak.rs](mailto:veljko.cocojevic@elfak.rs)
- Јован Јовановић [jovanovicjovan@elfak.rs](mailto:jovanovicjovan@elfak.rs)
- Андрија Грбушић [strucnjak@elfak.rs](mailto:strucnjak@elfak.rs)

Назив пројекта: **GalaxyUML**

## Садржај

|                                                          |    |
|----------------------------------------------------------|----|
| Контекст и циљ софтверског пројекта.....                 | 2  |
| Контекст.....                                            | 2  |
| Циљ софтверског пројекта.....                            | 2  |
| Архитектурно-специфични захтеви.....                     | 2  |
| Функционални захтеви.....                                | 2  |
| Нефункционални захтеви.....                              | 3  |
| Техничка и пословна ограничења.....                      | 4  |
| Техничка ограничења.....                                 | 4  |
| Пословна ограничења система.....                         | 4  |
| Архитектурни дизајн софтверског система.....             | 5  |
| Генерална архитектура система и структурни дијаграм..... | 11 |
| Бихевиорални дијаграм.....                               | 11 |
| Изабрани апликациони оквири и библиотеке.....            | 12 |

## Контекст и циљ софтверског пројекта

### Контекст

Пројекат се развија као интерактивна колаборативна десктопш апликација која омогућава корисницима да у реалном времену додају објекте на цртачку таблу и комуницирају путем чета. Апликација је намењена за онлајн састанке, предавања или тимски рад, где више корисника истовремено жели да ради на истој табли.

Систем функционише у дистрибуираном окружењу, са клијентима који се повезују на централни сервер (Blackboard) или користе message broker за рил-тајм синхронизацију.

Обезбеђује контролу приступа, тако да организатор састанка одређује ко и кад може да мења садржај табле.

### Циљ софтверског пројекта

Омогућити да више корисника симултано и у реалном времену цртају и раде на заједничкој табли. Имплементирати рил-тајм чет између корисника, како би могли да комуницирају током цртања.

Обезбедити перзистенцију промена и undo/redo функционалност, како би се сачувала историја рада и избегао губитак података.

Примена архитектурних образаца (MVVM/MVC, Blackboard, push type Event-based Implicit Invocation) ради модуларности, скалабилности и лаког одржавања.

Омогућити једноставно проширење система (нови објекти за цртање, додатне функционалности).

## Архитектурно-специфични захтеви

### Функционални захтеви

Сви корисници треба да могу да виде таблу и промене на њој у реалном времену. Организатор састанка има иницијално право измене изгледа

табле (цртање) и право давања дозволе осталим корисницима да мењају садржај.

Корисници могу да комуницирају путем заједничког чета.

Више различитих објеката који могу да се цртају (додају на таблу).

Постојање више дисјунктних група корисника где свака има приступ својој табли.

## Нефункционални захтеви

|                     |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                |
|---------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Перформансе         | Промене које начини један корисник треба да буду пропагиране до свих корисника за највише 2 секунде.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                           |
| Сигурност           | Логовање корисника. Сваки корисник има приступ само табли састанка ком је придружен.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                           |
| Управљање ресурсима | Ослобађање меморије након брисања објеката са табле.<br>Event processing не сме блокирати GUI.<br>Мрежни проток оптимизован; шаљу се само делта промене по догађају.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                           |
| Употребљивост       | Интуитивни GUI <ul style="list-style-type: none"><li>• Канвас и алати за цртање морају бити лако препознатљиви</li><li>• Дугмад јасно означена</li><li>• Чет прозор једноставан и прегледан</li></ul> Једноставна интеракција <ul style="list-style-type: none"><li>• Повлачење линија, селектовање и брисање објеката интуитивно</li><li>• Минималан број корака за основне операције</li></ul> Брза реакција система <ul style="list-style-type: none"><li>• Промене на табли и нове поруке се приказују брзо</li><li>• Не блокира се GUI thread</li></ul> Приступачност <ul style="list-style-type: none"><li>• Текст читљив, контраст боја добар</li></ul> |
| Доступност          | Теоретски систем може бити увек доступан (докле год сервер ради).                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                              |

|                 |                                                       |
|-----------------|-------------------------------------------------------|
| Поузданост      | Губљење порука није дозвољено.                        |
| Скалабилност    | Систем је предвиђен за 10-50 корисника.               |
| Модификабилност | Мора да обезбеди једноставно додавање нових објеката. |

## Техничка и пословна ограничења

### Техничка ограничења

Клијентска апликација мора бити имплементирана као desktop апликација, коришћењем C# и WPF технологије.

За организацију корисничког интерфејса и логике мора се применити MVVM архитектурни образац.

Комуникација између клијената и сервера мора бити реализована коришћењем message-passing библиотеке или message broker-a (нпр. ZeroMQ или RabbitMQ).

Серверска компонента мора подржавати event-based publish/subscribe комуникацију у push варијанти.

Систем мора бити компатибилан са Windows оперативним системом.

Перзистенција података мора бити реализована коришћењем централизованог repository-ја / базе података.

GUI нит не сме бити блокирана мрежном комуникацијом или обрадом догађаја (асинхрони рад).

### Пословна ограничења система

Систем је намењен едукативној и демонстрационој употреби у оквиру академског пројекта.

Обим функционалности је ограничен на:

заједничко цртање UML/дијаграмских елемената

текстуални chat

управљање улогама корисника

Максималан број истовремених корисника у једној сесији је ограничен на 10–50 корисника.

Аудио и видео комуникација нису део основне функционалности система (могућа будућа надоградња).

Систем не захтева интеграцију са спољним комерцијалним сервисима.

Безбедносни механизми су основног нивоа (аутентикација и контроле приступа), без напредних enterprise механизма.

Рок за испоруку архитектурне фазе је дефинисан наставним планом и временским оквиром курса.

## Архитектурни дизајн софтверског система

### 1. Архитектонски образци (целокупан систем):

- Client-server архитектура је присутна у целокупном систему, са React клијентом и ASP.NET Core сервером.
- Event-driven / publish-subscribe комуникација је имплементирана преко SignalR hub-а и real-time догађаја.
- Repository-based архитектура је присутна кроз EF Core DbContext и репозиторијуме.
- Clean Architecture у строгом облику није примењена, јер нема јасно раздвојене Domain / Application / Infrastructure / Presentation слојеве по правилима dependency inversion-а.

### 2. Обрасци пословне логике (домен):

- Domain model је богат by behavior: Team, Meeting, Diagram и други модели садрже пословна правила као што су Join, Leave, Ban, ChangeRole и StartMeeting.
- Ово је DDD-inspired приступ, али не и строго DDD са bounded contexts, aggregates, value objects и формалним application layer-ом.

### 3. Обрасци приступа подацима (Data Access):

- Repository pattern је примењен преко ITeamRepo, IUserRepo, IMeetingRepo и њихових имплементација.
- EF Core функционише као Data Mapper / ORM слој, што је у складу са Entity Framework Core моделом перзистенције.
- Hibernate није коришћен, јер се ради о .NET пројекту са EF Core, а не Java/JPA окружењу.
- Active Record образац није примењен у чистом облику.

#### **4. Обрасци за управљање стањем и трансакцијама (memorija / RAM):**

- Unit of Work је имплицитно присутан кроз ApplicationDbContext, који групише промене и чува их помоћу SaveChangesAsync().
- Ово обезбеђује једну логичку трансакцију над више ентитета у бази података.
- Identity Map није формално имплементиран као посебан објекат, али EF Core tracking механизам делује као основни механизам праћења ентитета у меморији.

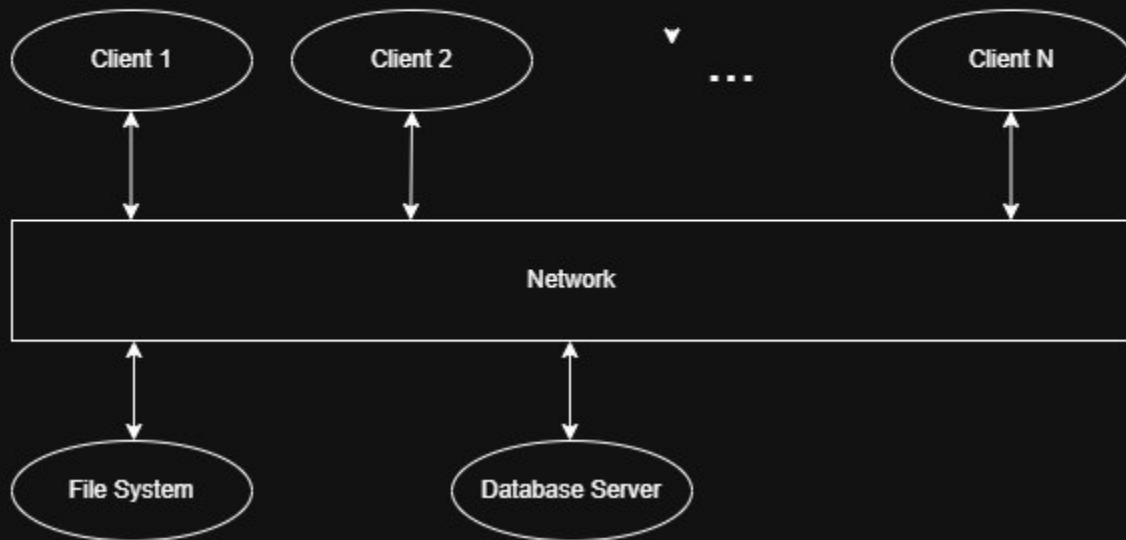
#### **5. Пројектни образци (GoF):**

- Observer-style pattern је присутан кроз SignalR event listener-е и callback-ове у React контекстима.
- Ово је пример event-driven реаговања, али не формално централно коришћење свих GoF образаца.

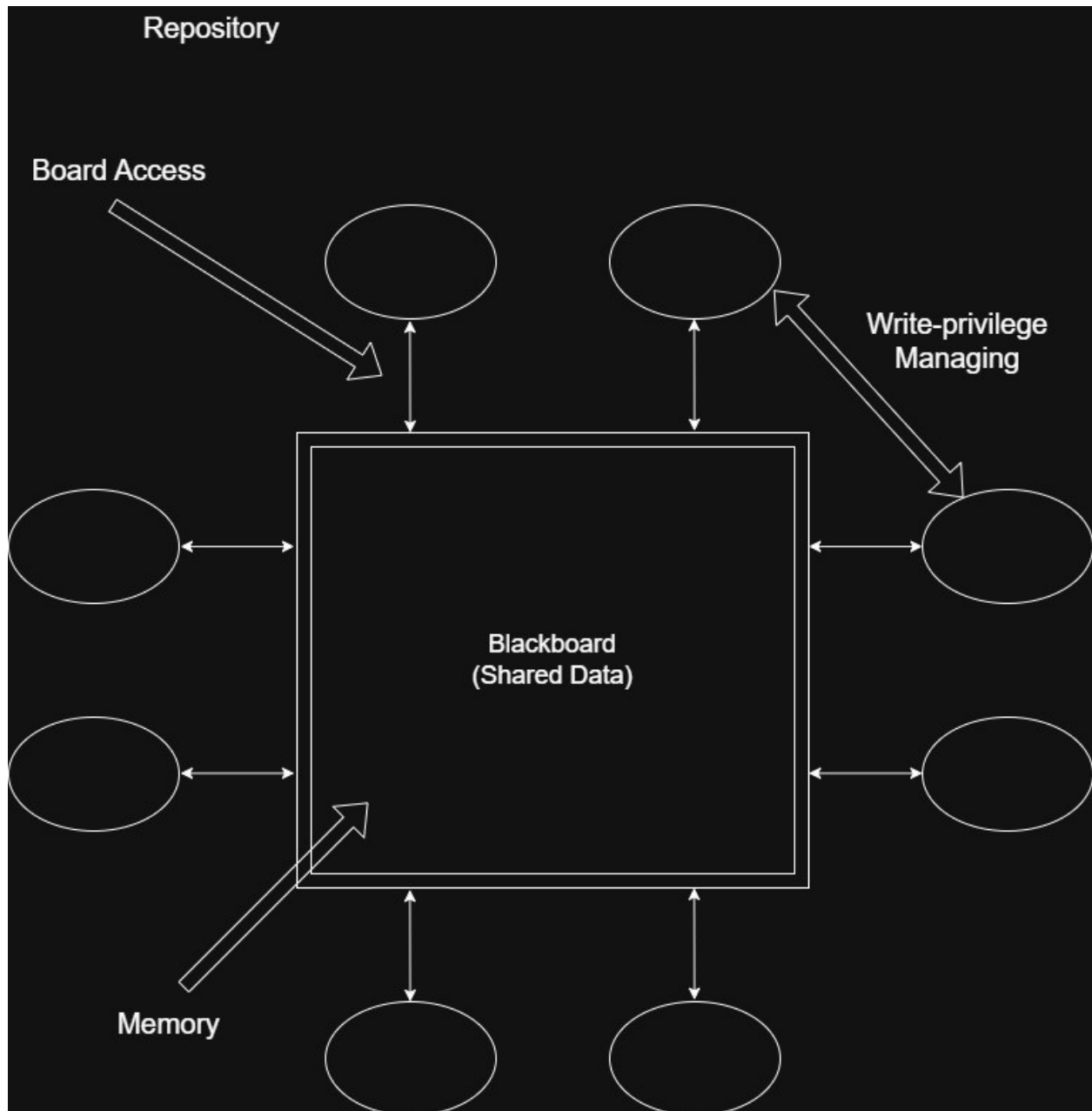
#### **Закључак:**

Најтачније описивање архитектуре је: pragmatic layered architecture са DDD елементима, EF Core repository/data mapper моделом и Unit of Work-ом кроз DbContext, али не чиста Clean Architecture ни strict DDD.

## CLIENT-SERVER

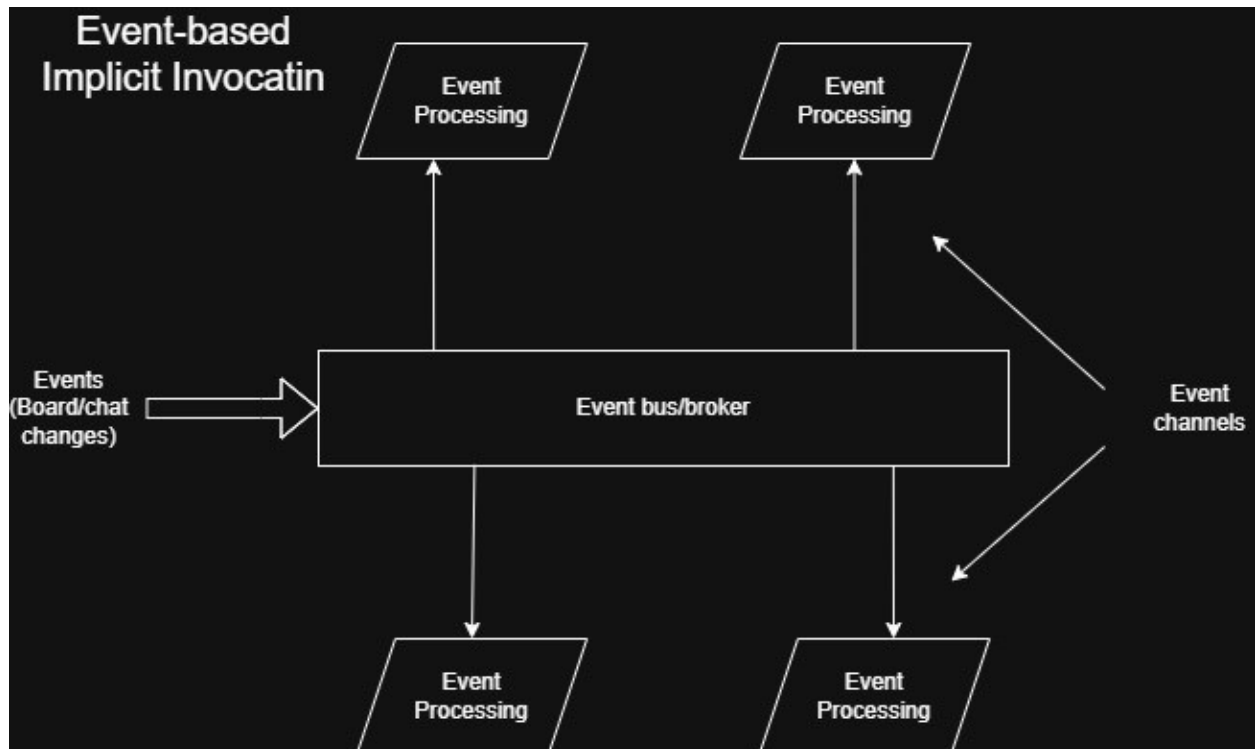


**Repository** – Образац за складиштење (дељење приступа). Радимо **Blackboard** (активно складиште).





**Event-based Implicit Invocation (push)** – Кад клијент начини промену на табли имплицитно се обавештавају сви остали клијенти који приступају истој табли. Радимо **Event-based** подваријанту и то **push** типа јер је циљ да се сви обавесте кад се деси промена и да такорећи цртачка табла одлучује кад ће се то десити (зато је **push**).

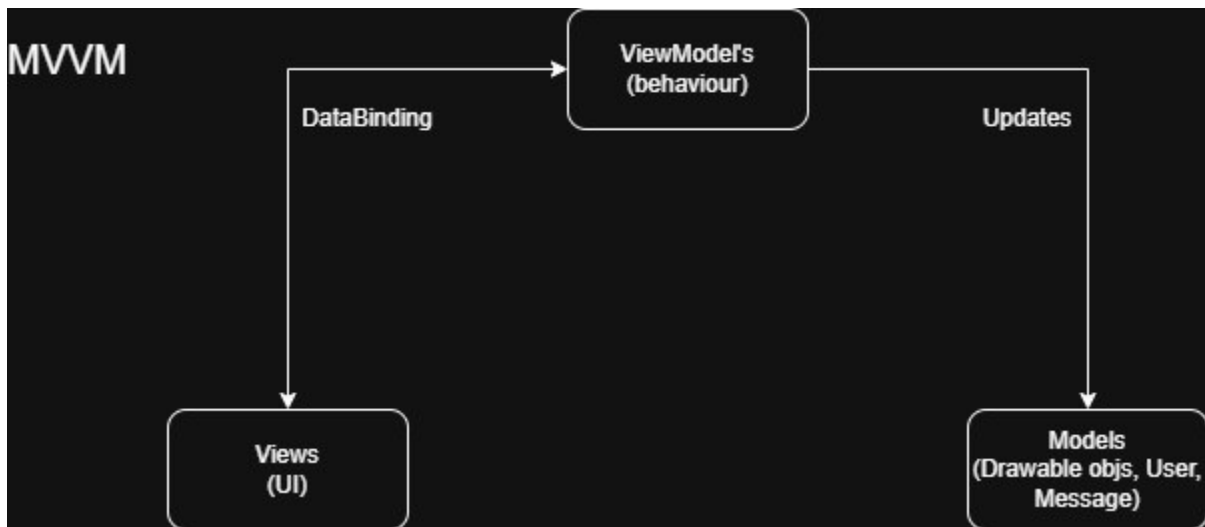


### MVVM-like

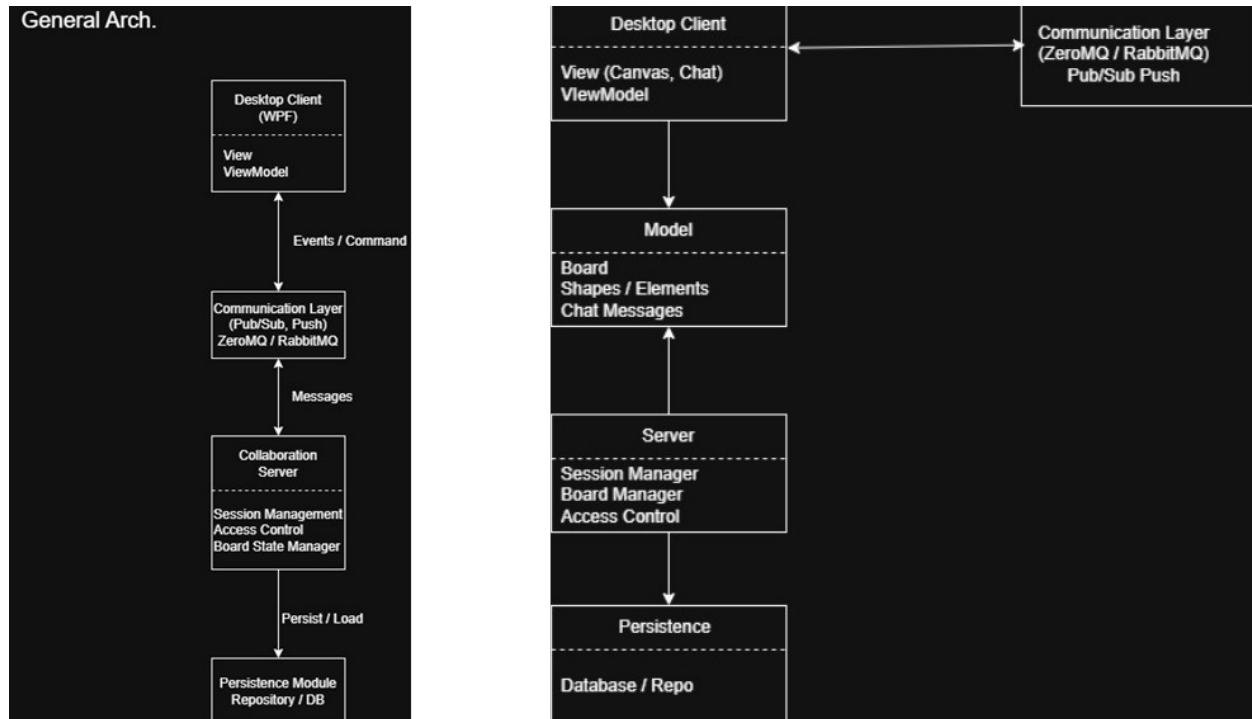
*Заправо React component-based архитектура са one-way data flow-ом.*

1. Корисник повуче линију на Canvas-у  
(акција мишем у View-у)
2. View региструје догађај и активира Command  
View не садржи пословну логику, већ позива DrawCommand дефинисан у ViewModel-у.
3. ViewModel прими Command  
ViewModel проверава да ли корисник има право цртања и припрема податке за нову линију.

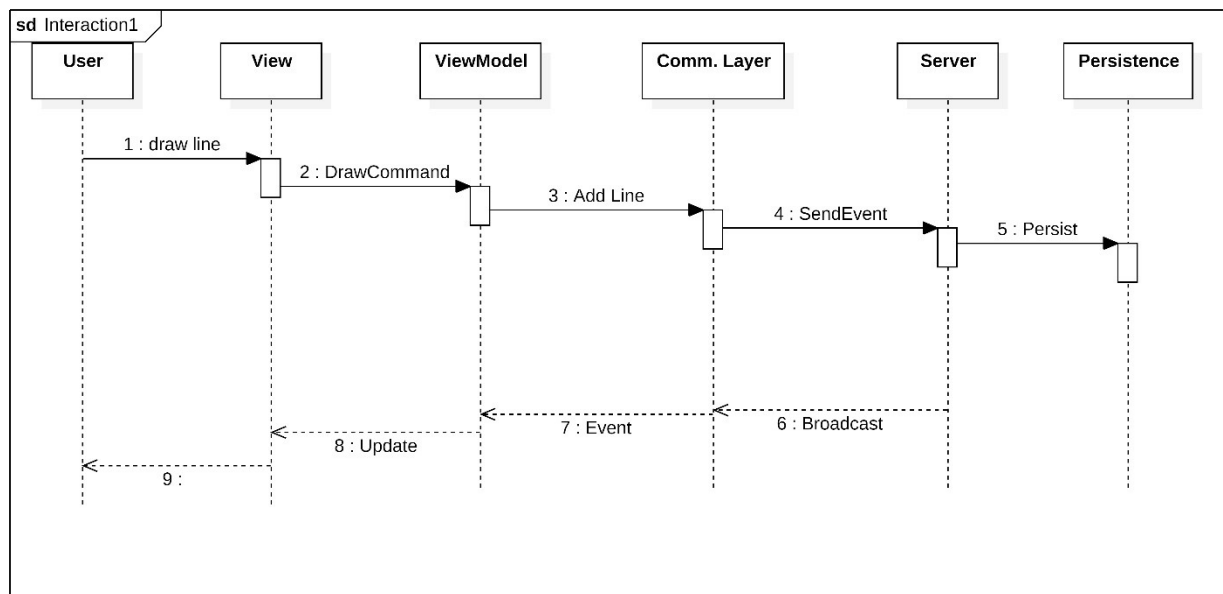
4. ViewModel мења Model  
Креира се нови објекат типа Line, који се додаје у ObservableCollection<Shape>.
5. ViewModel шаље догађај серверу  
Промена се пакује у event поруку и шаље преко Communication Layer-а (publish/subscribe, push варијанта).
6. Model је промењен → View се аутоматски освежава  
Захваљујући data binding механизму, View аутоматски приказује нову линију без ручног освежавања.
7. Остали клијенти добијају догађај (push)  
Њихов ViewModel ажурира Model, након чега се њихов View аутоматски освежава.



## Генерална архитектура система и структурни дијаграм



## Бихевиорални дијаграм



# Изабрани апликациони оквири и библиотеке

## 1. Клијентска компонента (web апликација)

**Језик:** TypeScript

**Framework:** React + Vite

### Разлог избора:

- Модеран и интерактиван кориснички интерфејс за UML моделовање са React компонентама, Canvas/SVG приказом и стилизованим контролама.
- Једноставна организација UI-ја кроз компоненте, context и hooks, што омогућава јасну сепарацију приказа и стања.
- Одлична подршка за јасан и предвидљив state flow, што омогућава аутоматско освежавање приказа при променама стања.
- Обрасци који се примењују на клијенту:

### Обрасци који се примењују у клијенту:

- Component-based UI образац:
- Component: приказује стање табле, chat порука и корисничких права.
- State/Context: чува и дистрибуира стање кроз апликацију и обрађује долазне event-е са сервера.
- View: Canvas, chat прозор и toolbar, који се аутоматски освежавају при променама у state-у.
- Event-based push:  
Клијент се претплаћује на догађаје са SignalR сервера и реагује када стигну нове промене.

### Библиотеке и алати:

- React – за компонентни кориснички интерфејс и управљање стањем.
- TypeScript – за типизовану имплементацију клијента.

- Vite, Tailwind CSS и @microsoft/signalr – за брз развој, стилизовање и real-time комуникацију.

## **2. Серверска компонента (дистрибуирани сервер / Blackboard)**

**Језик:** C#

**Тип апликације:** ASP.NET Core Web API + SignalR hub

**Обрасци који се примењују:**

- Client-server / Repository образац:  
Централизовано управљање стањем табле, историјом цртежа и chat порука кроз серверски API и базу података.
- Implicit Invocation / Event push:  
Сервер преко SignalR-а одлучује када ће клијентима послати нове догађаје.

**Библиотеке и алати:**

- ASP.NET Core SignalR – за дистрибуцију догађаја ка свим клијентима.
- Entity Framework Core и SQL Server – за перзистенцију података у централизованом бази.

## **3. Додатне напомене**

- React образац се примењује само на клијентску UI логику, док сервер користи ASP.NET Core + SignalR за real-time комуникацију.
- Оваква комбинација омогућава:
- Јасну модуларност и сепарацију одговорности
- Једноставно проширење система (додавање нових типова објеката или догађаја)
- Поуздану и real-time дистрибуцију догађаја свим корисницима